DG INTERNS HUB

WEEK 8: ADVANCED VAPT

Attack Chain & Full Exploitation Project

Target: OWASP Juice Shop (local training lab)
Environment: Kali Linux on VMware Workstation
Tool: Burp Suite Community Edition
Prepared by: Amar Pratap
Submission date: 28 October 2026

REPORT STATUS: PRELIMINARY — ATTACK CHAIN NOT VERIFIED

This report distinguishes observed behavior from confirmed vulnerabilities. No successful exploitation, account takeover, unauthorized data access, or privilege escalation is claimed without reproducible evidence.

Index / Table of Contents

Page numbers refer to the planned section starts in this document; update fields in Word if pagination changes.

Section	Contents	Page
1	Executive Summary	3
2	Introduction, Background and Objectives	4
3	Scope, Authorization and Rules of Engagement	5
4	Lab Environment, Architecture and Tools	6
5	Attack Surface Mapping and Application Flow	7
6	Testing Methodology and Evidence Management	8
7	Verified Observations and Technical Analysis	9
8	Preliminary Finding Assessment and Severity	10
9	Attack Chain Analysis and Validation Plan	11
10	Risk and Business Impact Analysis	12
11	Remediation and Retesting Strategy	13
12	Limitations, Conclusion and References	14
Appendix A	Evidence Register and Submission Checklist	15

Note: The table of contents is manually prepared for this version. If you add or remove pages, refresh the page numbers before submitting.

1. Executive Summary

This report documents a preliminary web application security assessment of a local OWASP Juice Shop instance running in Kali Linux under VMware Workstation. Burp Suite Community Edition was used to capture HTTP traffic and inspect requests. The project follows the DG Interns Hub Week 8 brief, which emphasizes attack-surface mapping, logic-based testing, chaining weaknesses, risk analysis, and remediation.

Only three observations are currently supported by the supplied evidence: (1) GET /rest/admin/application-version returned HTTP 200 OK and JSON version 20.3.0; (2) GET /rest/admin/application-configuration returned HTTP 304 Not Modified in one test; and (3) Burp HTTP history displayed traffic to the local Juice Shop instance.

Summary of status

Area	Status	Evidence / reason
Local target and traffic capture	Observed	Burp screenshots showed requests to 127.0.0.1:3000.
Version endpoint	Response verified	HTTP 200 with version 20.3.0.
Configuration endpoint	Inconclusive	HTTP 304 observed once; no sensitive body established.
IDOR / broken access control	Pending	No reproducible cross-account access result supplied.
Business logic	Pending	No workflow bypass or unauthorized state change supplied.
Attack chain	Not established	No two linked weaknesses and combined impact demonstrated.

Executive conclusion

The lab is functioning sufficiently to begin controlled testing, but the available evidence does not yet prove an exploitable vulnerability or a complete attack chain. The report therefore labels the version endpoint as a candidate for further assessment, treats the configuration endpoint as inconclusive, and keeps authorization/business-logic testing pending. Severity is not assigned without demonstrated impact.

Recommended next actions

- Repeat the configuration request without the conditional If-None-Match header and inspect the body.
- Capture baseline profile and basket requests from each controlled test account.
- Verify expected access controls using only objects belonging to the test accounts.
- Preserve redacted request/response evidence and update the finding status only after reproduction.

2. Introduction, Background and Objectives

2.1 What is VAPT?

Vulnerability Assessment and Penetration Testing (VAPT) combines the identification of potential weaknesses with controlled validation of whether those weaknesses can be exploited and what impact would result. A vulnerability scanner or unusual HTTP response may identify a lead, but manual verification is needed to distinguish a real security weakness from normal application behavior.

2.2 What makes this an advanced VAPT task?

The assignment focuses on relationships between weaknesses. An attack chain is a sequence in which an initial weakness enables or meaningfully assists a later weakness, leading to a larger effect. For example, an access-control weakness and a separate workflow-validation weakness might combine to expose or modify data. Such an example is a hypothesis only; this assessment does not claim that it occurred.

2.3 Project objectives

- Map pages, forms, APIs, and normal application workflows.
- Use Burp Proxy to capture requests and Repeater to replay controlled test cases.
- Investigate authorization, IDOR, parameter validation, business logic, and session behavior.
- Describe actual impact rather than inferring impact from endpoint names or status codes.
- Assign severity using exploitability and business impact, and recommend controls.
- Retest after fixes where source code or configuration is available.

2.4 Assessment approach

The workflow is scope confirmation → normal browsing and mapping → baseline capture → hypothesis-driven testing → impact verification → risk assessment → remediation → retest. The report marks incomplete stages as pending rather than implying completion.

3. Scope, Authorization and Rules of Engagement

Item	Definition
Target	OWASP Juice Shop running locally in the Kali Linux lab.
Host and port	http://127.0.0.1:3000, as used in the supplied commands.
Operating environment	Kali Linux in VMware Workstation.
Testing proxy	Burp Suite Community Edition.
Test accounts	Two fictional accounts were reported as created; individual response evidence was not supplied.
In scope	Manual, non-destructive testing against the local training target.
Out of scope	Public demo sites, third-party systems, real user data, destructive tests, denial of service, and systems without permission.

3.1 Authorization

This report assumes the tester owns or is explicitly authorized to test the local Juice Shop instance. The same techniques must not be applied to external systems without written authorization and a defined scope.

3.2 Safety rules

- Keep the Docker port bound to 127.0.0.1 and do not expose the intentionally vulnerable service to the internet or wider network.
- Use only fictional lab accounts and test records.
- Redact passwords, cookies, bearer tokens, JWTs, and other secrets from screenshots.
- Do not intentionally access unrelated users' records or change data outside the controlled lab.
- Stop a test if it causes unintended effects or crosses the approved scope.
- Use backups or disposable test data before testing state-changing business workflows.

3.3 Scope limitations

The report is based on the screenshots and outputs provided in the conversation. The full Burp project file, exported request history, server logs, and source-code review were not supplied. Conclusions must remain proportional to this evidence.

4. Lab Environment, Architecture and Tools

4.1 Components

Component	Role	Current evidence status
Kali Linux	Testing operating system	User identified Kali as the environment.
VMware Workstation	Runs the Kali VM	User identified VMware Workstation.
Docker	Runs the Juice Shop container	docker run returned a container ID.
OWASP Juice Shop	Intentionally vulnerable training app	Local endpoint returned a response; traffic appeared in Burp.
Burp Suite Community	Intercepts and replays HTTP requests	Screenshots showed HTTP history and Repeater views.
Browser	Generates ordinary application traffic	Used to browse local Juice Shop.

4.2 Local architecture

Browser → Burp Proxy → local Juice Shop at 127.0.0.1:3000. Docker maps the application's port to the loopback address so the service is available from the Kali machine only. Burp is used to observe requests, not to bypass authorization by itself.

4.3 Setup command observed

```
sudo docker run -d --name juice-shop -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

The command returned a container ID. A subsequent terminal screenshot showed the version endpoint responding. The current container status, image digest, and exact software build are not independently verified beyond the reported endpoint value.

4.4 Burp workflow

1. Open Proxy → HTTP history and generate traffic by using the application normally.
2. Select a relevant request and choose Send to Repeater.
3. Replay the unchanged request first to establish baseline behavior.
4. Change only one test condition at a time and compare status, response body, and application effect.
5. Save screenshots and redact session secrets before report inclusion.

4.5 Evidence storage

The user created ~/Week8_VAPT/ with screenshots, evidence, report, and presentation folders. The version response was saved through a terminal command to application-version.txt. The original screenshots should be inserted into the report before submission if required.

5. Attack Surface Mapping and Application Flow

Attack surface mapping records every page, form, API, and workflow that a user or browser can reach. The presence of an endpoint in Burp history proves that the endpoint was observed; it does not prove that the endpoint is vulnerable.

Observed path / feature	Evidence available	What to assess next
/rest/admin/application-version	HTTP 200; version 20.3.0	Confirm intended access policy and whether disclosure creates meaningful risk.
/rest/admin/application-configuration	HTTP 304 in one test	Repeat without If-None-Match; inspect actual response and access policy.
/api/Challenges/?name=Score%20Board	Shown in Burp history screenshot	Record normal response and whether data is public by design.
/rest/languages	Shown in Burp history screenshot	Document normal response and purpose.
/api/Quantities/	Shown in Burp history screenshot	Document normal response and purpose.
Registration / login	Two accounts reportedly created	Record normal login/registration flow; redact credentials.
Profile / basket	Requests reportedly captured	Compare each account's own data; verify authorization expectations.
Search / product pages	Normal browsing described	Record parameters, methods, response codes, and validation behavior.

5.1 Working application flow

Home page → product browsing/search → registration or login → profile and basket → checkout flow → API requests. This is a working map based on the actions described by the tester, not a complete route inventory.

5.2 Mapping record template

Field	Record
Action	What was clicked or submitted.
Request	HTTP method and path.
Authentication state	Logged out or test account A/B.
Parameters	Query, path, and body fields; redact secrets.
Response	Status, content type, relevant fields.
Expected behavior	What the application should permit.
Observed behavior	What actually happened.
Status	Observed / pending / verified finding / not reproduced.

6. Testing Methodology and Evidence Management

6.1 Baseline-first manual testing

6. Generate a request through a normal action in the local application.
7. Send a copy to Repeater while preserving the original request.
8. Replay the unchanged request and record the baseline status and response.
9. Change one non-destructive test condition at a time.
10. Compare the response and the actual effect in the application.
11. Repeat the test to rule out transient or caching effects.
12. Document the result, including negative results and limitations.

6.2 Authorization and IDOR methodology

IDOR (Insecure Direct Object Reference) is a form of broken access control where a user can access an object they are not authorized to access by manipulating or supplying an object reference. The object reference might be an identifier in a URL, request body, or API path. The core issue is missing server-side authorization, not merely the existence of a numeric or predictable ID.

- First record each test account's own profile/basket request and response.
- Determine the expected ownership rule.
- Use only test objects created by the tester when checking ownership enforcement.
- A finding requires a reproducible unauthorized result, not merely different responses or a changed status code.
- Redact cookies and authorization tokens from all evidence.

6.3 Business-logic testing

Business logic consists of application-specific rules: valid workflow order, quantity limits, checkout conditions, account ownership, and transaction state. Testing asks whether the server enforces these rules even when a request is replayed or a client value is changed. A user-interface restriction alone is insufficient if the server accepts an invalid state transition.

6.4 Evidence checklist

- Target host and request path visible.
- Baseline and modified requests distinguishable.
- Authentication context documented safely.
- Expected and observed behavior written clearly.
- Impact reproduced without touching unrelated data.
- Screenshots saved and secrets redacted.
- Retest recorded after remediation, if performed.

7. Verified Observations and Technical Analysis

Observation O-01 — Application version endpoint

Field	Observed value
Request	GET /rest/admin/application-version
Status	HTTP 200 OK
Content type	application/json
Response body	{"version":"20.3.0"}
Evidence	User-supplied terminal screenshot and saved-response command.

Interpretation: the endpoint returned version information under the tested condition, and the shown request did not visibly include an authentication header. This does not alone prove a vulnerability. The endpoint may intentionally provide version information. The security significance depends on the intended access policy and a demonstrated impact.

Observation O-02 — Application configuration endpoint

Field	Observed value
Request	GET /rest/admin/application-configuration
Status	HTTP 304 Not Modified in one test
Request context	If-None-Match was present in the earlier screenshot
Response body	No sensitive configuration body established
Evidence	User-supplied Burp screenshots.

Interpretation: 304 is a conditional HTTP caching response and commonly indicates that a cached representation has not changed. It is not evidence of configuration disclosure. The result of the proposed repeat request without If-None-Match was not supplied, so the endpoint remains inconclusive.

Observation O-03 — Local traffic capture

Burp history screenshots displayed HTTP requests to 127.0.0.1:3000 and paths including /api/Challenges, /rest/languages, and /api/Quantities. This supports the claim that local traffic was captured. It does not establish a vulnerability in those endpoints.

What is not verified

- No unauthorized access to another account's data.
- No successful business-logic bypass.
- No session reuse or logout weakness.
- No account takeover or privilege escalation.
- No complete attack chain.
- No remediation and retest result.

8. Preliminary Finding Assessment and Severity

Candidate C-01 — Potential version information disclosure

Attribute	Assessment
Status	Observed behavior; vulnerability not confirmed
Endpoint	/rest/admin/application-version
Observed effect	Returns version 20.3.0 with HTTP 200.
Potential concern	Version information can sometimes assist technology fingerprinting.
Confirmed impact	None established.
Severity	Not assigned; insufficient evidence.
Validation needed	Confirm intended access policy and demonstrate material security consequence.

Configuration endpoint

Do not list /rest/admin/application-configuration as a vulnerability based only on HTTP 304. A finding would require evidence of unauthorized access to sensitive configuration or another demonstrable security-control failure.

Severity framework

Severity	General guide — apply only after evidence
Critical	Severe compromise with broad, direct impact; typically little practical barrier.
High	Significant unauthorized access, control, or data impact with realistic exploitability.
Medium	Meaningful but more limited impact or additional prerequisites.
Low	Limited impact, narrow exposure, or difficult-to-exploit weakness.
Informational	Observation with no demonstrated security impact.

These are general descriptions, not assigned ratings for the current observations. Severity should consider required privileges, exploitability, confidentiality/integrity/availability impact, affected users, and business context. A status code alone is never a severity rating.

Finding lifecycle

Observed → hypothesis → reproduced → impact confirmed → severity assessed → remediation proposed → retested. A finding that cannot be reproduced should be marked not reproduced or pending, not reported as a confirmed exploit.

9. Attack Chain Analysis and Validation Plan

The Week 8 assignment's main objective is to show how multiple weaknesses combine into a larger impact. Current evidence does not support a successful chain. The sequence below is a validation plan, not a claim that exploitation occurred.

Stage	Status	Evidence required
1. Entry point	Observed	Local endpoint and request captured.
2. First weakness	Pending	Reproducible failure of an expected security control.
3. Second weakness	Pending	Independent weakness that combines with the first.
4. Combined impact	Not demonstrated	Safe, repeatable proof of unauthorized effect.
5. Remediation and retest	Pending	Evidence that the fix blocks the relevant step(s).

Safe candidate investigation

Compare authorization behavior for the two controlled accounts in their own profile and basket workflows. If an API request references an object, verify the ownership rule using only test objects created by the tester. Do not access unrelated data or swap credentials without a clear, controlled plan.

Attack-chain acceptance criteria

- Each weakness has an independent reproducible proof of concept.
- The dependency between weaknesses is demonstrated.
- The combined effect is materially greater than the isolated observations.
- The report includes original request/response evidence and exact conditions.
- Testing stays within the local lab and uses only test data.

Example diagram for the report

Observed entry point → [Weakness 1: pending] → [Weakness 2: pending] → [Combined impact: not demonstrated] → Remediation and retest.

Required statement for current status: “A complete attack chain was not established in the evidence available for this submission.”

10. Risk and Business Impact Analysis

Risk analysis separates confirmed impact from potential impact. Business consequences should be discussed conditionally until the weakness is reproduced.

Area	Potential impact if confirmed	Current status
Version information	May aid fingerprinting if disclosure is not intended and supports a realistic exploit path.	Impact not established.
Configuration access	Could expose internal settings if sensitive configuration is returned without authorization.	Not a finding; only 304 observed.
IDOR / object authorization	Could expose or modify another user's records.	Pending; no unauthorized access demonstrated.
Business logic	Could allow invalid workflow transitions or unauthorized transactions.	Pending; no bypass demonstrated.
Session management	Could allow unauthorized session reuse if controls fail.	Pending; no result supplied.

10.1 Confidentiality, integrity and availability

- Confidentiality: unauthorized reading of personal, account, or transaction information.
- Integrity: unauthorized changes to profiles, baskets, orders, or other records.
- Availability: disruption or loss of service; no availability testing was performed.
- Trust and compliance: potential loss of user confidence if a real deployment were affected.

10.2 Business impact statement

No real customer data, financial loss, account takeover, or production impact has been demonstrated. Those impacts should not be written as actual outcomes. If a later test verifies unauthorized access, document exactly which test object was accessed, which account was authorized, and what the server returned.

10.3 Risk treatment

Prioritize verified high-impact authorization failures, then business-logic and session weaknesses according to demonstrated likelihood and impact. Informational observations should not distract from confirmed security-control failures.

11. Remediation and Retesting Strategy

11.1 Access control and IDOR

- Perform authorization checks server-side on every sensitive operation.
- Bind requested objects to the authenticated principal and permitted role.
- Do not rely on hidden buttons, client-side checks, or unguessable identifiers alone.
- Add regression tests proving Account A cannot read or change Account B's test objects unless explicitly permitted.

11.2 Business logic and parameter tampering

- Validate workflow state and transition order on the server.
- Recalculate prices, discounts, totals, and permissions on the server rather than trusting client values.
- Validate quantities, object ownership, and allowed field values at the point of use.
- Reject unexpected fields or state transitions where appropriate and log relevant security events.

11.3 Session management

- Use secure session handling and appropriate expiration.
- Invalidate sessions on logout and after sensitive security events.
- Rotate session identifiers after authentication or privilege changes where appropriate.
- Never include live session secrets in screenshots or reports.

11.4 Information disclosure and caching

Decide whether the version endpoint should be public. If version disclosure is unnecessary, restrict the endpoint or return only required information. For configuration endpoints, enforce authorization and avoid caching sensitive responses in shared or inappropriate contexts. Cache headers should be selected based on data sensitivity and application design.

11.5 Retest plan

13. Record the original proof of concept and expected failure condition.
14. Apply the fix in a controlled lab environment.
15. Repeat the exact request and workflow.
16. Verify the unauthorized effect no longer occurs.
17. Run regression checks for legitimate users.
18. Save before/after evidence and record any remaining limitations.

No fix or retest result was supplied for this report. The controls above are recommendations, not verified remediation.

12. Limitations, Conclusion and References

12.1 Limitations

- Assessment is based on user-supplied outputs and screenshots, not independent forensic collection.
- Full Burp request history and raw HTTP exchanges were not provided.
- The modified configuration request result is not available.
- No reproducible IDOR, business logic bypass, session weakness, privilege escalation, or complete chain was supplied.
- No source-code change or remediation retest was supplied.
- Original screenshots are not embedded in this generated report and should be added before final submission if required.

12.2 Conclusion

The local Juice Shop environment was started using Docker, and Burp Suite captured local application traffic. The version endpoint returned HTTP 200 with version 20.3.0. The configuration endpoint returned HTTP 304 in one test. These observations do not prove an exploitable vulnerability or successful attack chain.

The appropriate next step is to complete controlled authorization and business-logic tests, preserve exact request/response evidence, confirm the intended access policy, and update this report only with reproducible results. Transparent reporting of pending tests is preferable to claiming an exploit that has not been demonstrated.

12.3 References

- OWASP Juice Shop: https://owasp.org/www-project-juice-shop/
- OWASP Juice Shop source repository: https://github.com/juice-shop/juice-shop
- PortSwigger Burp Suite Repeater: https://portswigger.net/burp/documentation/desktop/tools/repeater
- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Top 10: https://owasp.org/www-project-top-ten/

Appendix A. Evidence Register and Submission Checklist

A.1 Evidence register

ID	Evidence item	What it supports	Limitations
E-01	Burp HTTP history screenshot	Local requests and observed paths	Does not prove endpoint vulnerability.
E-02	Version endpoint response screenshot	HTTP 200 and version 20.3.0	Access policy and impact unconfirmed.
E-03	Configuration endpoint screenshot	HTTP 304 in one test	No sensitive response body established.
E-04	Docker run output	Container launch command returned an ID	Does not independently prove current health.
E-05	Account/profile/basket requests	Reported captured by user	Detailed responses not supplied for this report.

A.2 Final submission checklist

- Insert original screenshots into the relevant report sections.
- Ensure each screenshot is readable and redacts credentials/tokens.
- Update table of contents page numbers if pagination changes.
- Confirm name, internship title, target, date, and organization details.
- Do not call a candidate observation a confirmed vulnerability without proof.
- Keep the attack chain marked pending unless two linked weaknesses and impact are reproduced.
- Ensure PPT claims match the report.
- Review the LinkedIn post so it describes only work actually completed.

A.3 Report status

Document status: Preliminary assessment with verified observations and clearly marked pending tests. This is not a claim of full exploitation.